

TelemetriaMqtt v2 — Checkpoint 8

Firmware — parser CAN (TWAI)

Projeto de Telemetria FSAE

18 de setembro de 2026

Abstract

Este documento detalha o oitavo checkpoint do projeto: a primeira parte do firmware do ESP32, que lê os quadros CAN do PDM32 e os decodifica nos 20 sinais de telemetria. A decodificação foi isolada numa biblioteca em C++ puro, totalmente testada sem hardware, e a leitura do barramento ficou numa camada fina que só a compilação verifica. O documento registra com clareza o que está provado por teste e o que só será provado quando o hardware chegar.

Contents

1	Contexto e objetivo	1
2	Decisão de arquitetura: separar decodificação e hardware	1
3	Estrutura adicionada	2
4	O contrato: tabela de payloads	2
5	O parser	2
5.1	Interface	2
5.2	Decodificação byte a byte	3
5.3	Política para quadros inválidos	3
6	A camada TWAI (só hardware)	4
7	Testes	4
7.1	Resultado da execução	4
7.2	Como o TDD correu de fato	5
8	O que NÃO está verificado	5
9	Validação manual	5
10	Decisões para você revisar	6
11	Nota sobre metodologia	6
12	Resumo do que ficou pronto	6
13	Próximos passos	6

1 Contexto e objetivo

Até o checkpoint 7 todo o caminho *depois* do dispositivo estava pronto e testado: broker, ingestão, InfluxDB e os dois dashboards, alimentados por um mock em Python que finge ser o ESP32. Este checkpoint começa o dispositivo de verdade. O ESP32 é ligado ao CAN2 do carro por um transceiver SN65HVD230; o PDM32 transmite ali 10 quadros (IDs 0x100 a 0x131, definidos no Race Studio) e o firmware precisa transformá-los nos mesmos 20 sinais que o mock já publica.

O escopo aqui é **só a leitura e a decodificação**. Montar o JSON, WiFi, MQTT e TLS ficam para o checkpoint 9.

2 Decisão de arquitetura: separar decodificação e hardware

O ponto central do desenho é dividir o código em duas partes com garantias diferentes:

Parte	Responsabilidade	Como é verificada
lib/can_parser	Converter um quadro (id, dlc, bytes) em valores dos sinais. C++17 puro, sem Arduino nem ESP-IDF.	Testes nativos (Unity), no PC.
src/can_twai.*	Instalar o driver TWAI e entregar os quadros recebidos ao parser. Só existe no alvo esp32dev.	Apenas compilação.

A razão é a metodologia do projeto: TDD obrigatório e nenhum checkpoint fechado com testes falhando. Tudo o que tem lógica (offsets, ordem dos bytes, validações) precisa poder ser testado sem o ESP32. Empurrando essa lógica para uma biblioteca sem dependências de plataforma, a camada que depende do driver fica tão fina que há pouco onde um bug se esconder.

3 Estrutura adicionada

Listing 1: Arquivos novos e alterados

```
1  firmware/
2    lib/can_parser/
3      can_parser.h          (novo: CanFrame, Telemetry, parse_frame)
4      can_parser.cpp        (novo: decodificação)
5    src/
6      can_twai.h            (novo)
7      can_twai.cpp          (novo: driver TWAI, só esp32dev)
8      main.cpp              (alterado: lê, decodifica, imprime na serial)
9    test/test_can_parser/
10     test_main.cpp          (novo: 20 testes)
```

4 O contrato: tabela de payloads

O parser implementa exatamente a tabela “Plano de payloads” do SCOPE.md, que é o contrato entre a configuração do Race Studio e o firmware. Todos os valores são little endian; os float são IEEE 754 de 32 bits.

ID	Freq	DLC	Bytes 0–3	Bytes 4–7
0x100	20 Hz	8	rpm	throttlePosition
0x101	20 Hz	8	lambda	map
0x110	10 Hz	8	gpsSpeed	shifterCurrent
0x111	10 Hz	8	bicosD1Current	bicosD2Current
0x112	10 Hz	8	bobinasCurrent	poTotCurrent
0x113	10 Hz	8	poTotCrntAll	partidaCurrent
0x114	10 Hz	4	extBattery	—
0x120	5 Hz	8	ventoinhaCurrent	bombaCurrent
0x130	2 Hz	8	engineTemp	airTemp
0x131	2 Hz	8	brakeLightCurrent	bombaInput (uint16, bytes 4–5) e giratoriaInput (uint16, bytes 6–7)

Os dois inputs digitais são uint16 e não float porque só valem 0 ou 1 (Momentary/Toggle), como registrado no SCOPE.md.

5 O parser

5.1 Interface

Listing 2: firmware/lib/can_parser/can_parser.h (resumo)

```

1 struct CanFrame {
2     uint32_t id;
3     uint8_t dlc;
4     bool extended;           // true = ID de 29 bits
5     uint8_t data[8];
6 };
7
8 struct Telemetry {
9     float rpm, throttlePosition, lambda, map;    // 0x100, 0x101
10    float gpsSpeed, shifterCurrent, ...;         // 0x110..0x114
11    // ... 18 floats no total
12    uint16_t bombaInput;
13    uint16_t giratoriaInput;
14 };
15
16 bool parse_frame(const CanFrame &frame, Telemetry &out);

```

Os campos de Telemetry têm **exatamente os mesmos nomes** das chaves do JSON que o `mock_publisher` envia. No checkpoint 9 o firmware poderá serializar a estrutura direto para o mesmo contrato, sem tabela de tradução. Ela guarda os *últimos valores conhecidos*: cada quadro atualiza só os campos do seu payload e deixa os outros como estavam. Isso importa porque os payloads chegam em frequências diferentes (de 20 Hz a 2 Hz).

A estrutura foi feita sem *padding* de propósito (18 float e 2 uint16, 76 bytes). Assim os testes conseguem comparar duas instâncias byte a byte e provar que um quadro inválido não alterou nada; um `static_assert` nos testes garante essa propriedade.

5.2 Decodificação byte a byte

Listing 3: Leitura de um float little endian

```

1 float read_f32_le(const uint8_t *p) {
2     const uint32_t bits = static_cast<uint32_t>(p[0])
3                          | (static_cast<uint32_t>(p[1]) << 8)
4                          | (static_cast<uint32_t>(p[2]) << 16)
5                          | (static_cast<uint32_t>(p[3]) << 24);

```

```

6  float value;
7  std::memcpy(&value, &bits, sizeof(value));
8  return value;
9  }

```

Em vez de converter o ponteiro para `float*`, os quatro bytes são montados num inteiro e só então reinterpretados com `memcpy`. Isso dá o resultado correto em qualquer endianness do processador e evita leitura desalinhada. O ESP32 e o PC x86 dos testes são ambos little endian, mas o código não depende disso.

5.3 Política para quadros inválidos

O protocolo não diz o que fazer com um quadro que não se encaixa, então esta é uma decisão de projeto, registrada no `SCOPE.md` e aberta a revisão:

Situação	Motivo	Resultado
ID fora da tabela	Não é um payload do PDM configurado.	Rejeita
Quadro estendido (29 bits)	O PDM foi configurado com IDs de 11 bits; um estendido com o mesmo número é outro endereço, de outro dispositivo.	Rejeita
DLC menor que o do contrato	Não há bytes suficientes para ler os campos.	Rejeita
DLC maior que o do contrato	Os bytes do contrato continuam no mesmo lugar (por exemplo, <code>0x114</code> configurado com 8 em vez de 4).	Aceita, ignora o resto

Quando rejeita, o parser **não escreve nada** em **Telemetry**: a validação completa (estendido, ID, DLC) acontece antes de qualquer atribuição, então não existe caminho que devolva **false** depois de ter alterado um campo. Uma falha de leitura nunca deixa a telemetria meio atualizada.

6 A camada TWAI (só hardware)

`can_twai.cpp` instala o driver, lê os quadros da fila e os converte em `CanFrame`. Tem 49 linhas e nenhuma lógica de decodificação. Três decisões merecem destaque.

- **Modo NORMAL e não LISTEN_ONLY.** O `SCOPE.md` registra que nada mais usa o CAN2 do carro. Se o ESP32 for o único receptor, ninguém daria *ACK* aos quadros do PDM, e o PDM enxergaria erro em todo quadro, retransmitindo sem parar. No modo normal o controlador só dá *ACK*. O firmware nunca chama `twai_transmit()`, então não escreve dados no barramento. Se no carro existir outro nó dando *ACK* no CAN2, `LISTEN_ONLY` passa a ser a opção mais segura. Isso só se confirma com hardware.
- **Pinos provisórios: TX = GPIO21, RX = GPIO22.** São os do exemplo oficial do ESP-IDF, fora dos pinos de boot. Precisam ser conferidos contra a fiação real do SN65HVD230; são duas constantes no início do arquivo.
- **Filtro aceita tudo, leitura sem bloqueio.** Quem descarta o que não é da tabela é o parser, que já é testado. `twai_receive` usa timeout zero para não travar o `loop()` se a fila estiver vazia, e quadros de requisição remota (sem dados) são ignorados.

O `main.cpp` lê todos os quadros pendentes a cada volta, chama o parser e, uma vez por segundo, imprime na serial contadores de quadros aceitos e ignorados mais alguns valores (`rpm`,

`engineTemp`, `extBattery`). É só uma ferramenta de conferência na bancada até o checkpoint 9.

7 Testes

Vinte testes novos em `firmware/test/test_can_parser/test_main.cpp` (Unity, ambiente `native`), agrupados assim:

Grupo	Testes	O que provam
Estado inicial	1	<code>Telemetry{}</code> começa zerada.
Um teste por payload	10	Cada ID decodifica seus campos com os valores certos, incluindo um número negativo (<code>airTemp</code>) e o quadro de 4 bytes (<code>0x114</code>).
Ordem dos bytes	1	Os <code>uint16</code> são little endian (<code>0x0100</code> e <code>0xABCD</code>); um 0 ou 1 sozinho não distinguiria.
Composição	3	Um quadro mexe só nos seus campos; os 10 payloads juntos preenchem os 20 sinais; um quadro novo sobrescreve o valor anterior.
Entradas inválidas	5	ID desconhecido, DLC curto (para os 10 payloads), DLC zero, DLC maior que o contrato e quadro estendido. Os rejeitados provam que <code>Telemetry</code> não mudou.

Os bytes esperados foram calculados de forma independente do parser, com `struct.pack` do Python, e escritos como literais nos testes (por exemplo, 1234.5 vira 00 50 9A 44). Se os testes reusassem a lógica do parser para gerar a expectativa, um erro de offset ou de ordem passaria despercebido nos dois lados.

7.1 Resultado da execução

Listing 4: Saída real de `pio test -e native` (resumo)

```
1 native:test_scaffolding [PASSED]
2 native:test_can_parser  [PASSED]
3 ===== 21 test cases: 21 succeeded in 00:00:00.793
   =====
```

O alvo de hardware compila sem avisos: `pio run -e esp32dev` usa 20,9% da flash (273 649 de 1 310 720 bytes) e 6,6% da RAM (21 592 de 327 680 bytes).

7.2 Como o TDD correu de fato

1. **Vermelho.** Os 20 testes foram escritos antes de existir qualquer código do parser. A execução falhou na compilação com `can_parser.h`: `No such file`: o motivo certo, o módulo ainda não existia.
2. **Verde.** Com o parser implementado, os 20 passaram na primeira execução.
3. **Teste de mutação.** Passar de primeira não prova que os testes conseguem falhar. Por isso o parser foi **quebrado de propósito** de três formas, uma de cada vez, e a suíte precisou detectar cada uma. O código original foi restaurado e reconferido (21 de 21).

Mutação	Testes que a detectaram	Falhas
uint16 lido em big endian	0x131, ordem dos bytes, 10 payloads juntos	3
giratoriaInput lido do offset 4 em vez de 6	0x131, ordem dos bytes	2
Checagem de DLC removida	DLC curto, DLC zero	2

Um detalhe honesto sobre a segunda mutação: o teste “10 payloads juntos” *não* a detectou, porque nele `bombaInput` e `giratoriaInput` valem ambos 1, então ler do offset errado dá o mesmo resultado. Quem pega esse erro são os dois testes dedicados ao 0x131. É uma fraqueza pontual desse teste composto, sem consequência para a cobertura geral, mas vale saber que ele sozinho não bastaria.

Nenhum teste anterior foi alterado neste checkpoint.

8 O que NÃO está verificado

- **O TWAI realmente receber quadros do PDM.** `can_twai.cpp` e `main.cpp` têm garantia de *compilar*, e nada além disso. Que o controlador sobe a 500 kbps, que os pinos estão certos e que o ACK funciona só se prova com o ESP32, o transceiver e o PDM (ou outro nó transmitindo).
- **A camada física do CAN** (terminação, cabo, nível de tensão), já registrada no `SCOPE.md` como fora do escopo até haver hardware.
- **O CI não compila o alvo esp32dev.** Ele roda só os testes nativos, então um erro de compilação em `can_twai.cpp` passaria batido lá. Adicionar um passo de compilação custaria alguns minutos por execução (baixa o toolchain do ESP32); fica como decisão sua.

9 Validação manual

Não há validação manual possível hoje: depende de hardware que ainda não chegou. O que dá para fazer agora é `cd firmware && pio test -e native` (deve dar 21 de 21) e `pio run -e esp32dev` (deve compilar), além de ler o diff.

Roteiro para quando o hardware chegar (não executado):

1. Ligar o SN65HVD230 aos pinos definidos em `can_twai.cpp` e ao CAN2, e gravar o firmware.
2. Abrir a serial (`pio device monitor`) e conferir que o contador `ok` cresce, que `ignorados` fica próximo de zero e que `rpm`, `engineTemp` e `extBattery` acompanham o carro.
3. Se `ok` ficar em zero, suspeitar primeiro dos pinos, da velocidade (500 kbps), do modo NORMAL contra LISTEN_ONLY e da configuração dos payloads no Race Studio (IDs de 11 bits, DLC, “Save” e “Transmit”).

10 Decisões para você revisar

1. **Modo NORMAL do TWAI** (em vez de LISTEN_ONLY), pelo motivo do ACK explicado acima.
2. **Pinos GPIO21 e GPIO22**, a confirmar contra a fiação.
3. **Política de quadros inválidos**, em especial aceitar DLC maior que o do contrato.
4. **Compilar o alvo esp32dev no CI**: vale o custo em minutos por execução?

11 Nota sobre metodologia

O ciclo foi o do projeto: testes primeiro, vermelho pelo motivo certo, implementação, verde, e depois o teste de mutação para provar que os testes conseguem falhar. As escolhas de tecnologia foram registradas no `SCOPE.md` antes de fechar o checkpoint: o driver TWAI do ESP-IDF (já incluído no core Arduino-ESP32, sem biblioteca extra) e o parser em C++17 puro. O que não pôde ser testado foi separado do que pôde e nomeado como tal, em vez de ser tratado como coberto.

12 Resumo do que ficou pronto

- Biblioteca `can_parser` que decodifica os 10 payloads do PDM nos 20 sinais, sem dependência de plataforma.
- 20 testes nativos, com bytes esperados calculados de forma independente e validados por teste de mutação.
- Camada TWAI e `main.cpp` que compilam para o ESP32 sem avisos.
- Política explícita e testada para quadros inválidos: nunca deixam a telemetria meio atualizada.

13 Próximos passos

Este checkpoint espera a sua revisão. Se aprovado, o commit fecha o checkpoint com a tag `v0.8.0`. O próximo é o **checkpoint 9**: WiFi, MQTT com TLS e a publicação do JSON no tópico `telemetria/esp32/data`, usando o `Telemetry` deste checkpoint como fonte, começando pelos testes.